Universidad del Valle de Guatemala
SISTEMAS Y TECNOLOGIAS WEB

Laboratorio #4

Fecha de Entrega: 28 de abril de 2026

Descripción:

En este laboratorio construiremos una API REST funcional desde cero usando Node.js y Express. El objetivo sera depurar un servidor HTTP nativo con errores, a diseñar e implementar un CRUD completo para gestionar una colección de datos de su elección. Al finalizar, tendrán una API que podrás consumir desde cualquier cliente (Postman, navegador, o tu propio frontend con fetch).

Entregables:

- **El código fuente** en un **repositorio Git**.

Materiales: Ejemplos de template de apis

Recursos Recomendados:

- Documentación oficial de Express
- HTTP Status Codes — MDN
- REST API Design Best Practices
- Postman Learning Center
- Node.js ES Modules Documentation

Estructura recomendada:

```
mi-laboratorio-6/
├── servidor-malo.js          ← Archivo original (sin modificar)
├── servidor-corregido.js     ← Tu versión corregida
├── datos.json                ← Datos para el servidor HTTP
├── api/
│   └── index.js              ← Tu API con Express (Parte 2)
├── package.json
├── SOLUCION.md               ← Documentación de errores (Parte 1)
├── PRUEBAS.md                ← Evidencia de pruebas con Postman
├── screenshots/              ← Capturas de pantalla
└── README.md                 ← Descripción general del proyecto
```

Antes de Empezar:

require vs import

- *Dos formas de usar módulos en Node.js*
- Hasta ahora hemos visto en clase el uso de require() para importar módulos. Esta es la sintaxis de CommonJS (CJS), el sistema de módulos original de Node.js. Sin embargo, JavaScript moderno usa ES Modules (ESM) con import/export, que es el mismo sistema que ya conocemos del navegador.

Característica	CommonJS (`require`)	ES Modules (`import`)
Sintaxis	`const express = require("express")`	`import express from "express"`
Carga	Síncrona (se ejecuta línea por línea)	Asíncrona (el motor optimiza la carga)
Disponibilidad	Funciona por defecto en Node.js	Requiere configuración (ver abajo)
Uso en navegador	No funciona en el navegador	Funciona nativamente en `<script type="module">`
Exportar	`module.exports = { ... }`	`export default ...` o `export { ... }`

Cuidado con la “s” la de de module.exports

¿Cómo habilitar import en Node.js?

Node.js usa CommonJS por defecto. Para usar import, tienen dos opciones:

Opción A — Cambiar la extensión del archivo a .mjs:

- mi-servidor.mjs ← Node.js lo trata como ES Module automáticamente

Opción B — Agregar "type": "module" en el package.json (recomendado):

```
{ "name": "mi-proyecto", "version": "1.0.0", "type": "module", "dependencies": { "express": "^4.21.2" } }
```

Con esta línea, todos los archivos .js del proyecto se tratan como ES Modules.

Comparación:

CommonJS (modules)

```
//archivo: app.js
const express = require("express");
const { uid } = require("uid"); c
onst app = express();
module.exports= app;
```

ES Modules (la forma moderna):

```
// archivo: app.js (con "type": "module" en package.json)
import express from "express";
import { uid } from "uid";
constapp = express();
export default app;
```

Regla de oro: No mezclen require e import en el mismo archivo. Escogan un sistema y usenlo consistentemente en todo el proyecto. Para este laboratorio pueden usar el que prefieran, pero si el archivo original usa import, asegúrate de tener "type": "module" en el package.json.

Instrucciones:

Parte 1: Depuración — El Servidor Roto (35 pts)

- Creen un nuevo proyecto con npm init -y.
- Asegúrense de configurar tu package.json correctamente para que el archivo pueda ejecutarse (recuerden la sección anterior sobre módulos).
- Creen un archivo datos.json en la raíz de tu proyecto con contenido válido,
    - por ejemplo:

```
{
"nombre": "Tu Nombre",
"carnet": "241XX",
"curso": "Sistemas y Tecnologías Web"
}
```

- Ejecuten el servidor con node servidor-malo.js.
- Identifiquen cada error, corríganlos, y documentaen el proceso

¿Qué buscar?

Presten atención a estas categorías de errores:

- Errores de sintaxis: ¿El archivo se puede ejecutar sin que Node.js lance un error?
- Errores de lógica asíncrona: ¿Se están manejando correctamente las promesas?
- Errores de protocolo HTTP: ¿Los Content-Type son correctos?
- Errores de respuesta: ¿Los códigos de estado HTTP son apropiados para cada situación?

Entregable de la Parte 1

- Un archivo SOLUCION.md con la siguiente estructura por cada error:

```
### Error #N: [Nombre descriptivo del error]
**Ubicación:** Línea X del archivo original
**Tipo de error:**[Sintaxis / Lógica / HTTP / Asincronía]
**Qué estaba mal:** [Explicación clara] *
*Cómo lo corregí:** [Código antes → código después]
**Por qué funciona ahora:** [Justificación técnica breve]
```

Parte 2: Evolución — De Servidor HTTP a API con Express (50 pts)

Ahora que entienden cómo funciona un servidor HTTP nativo, es momento de construir algo real con Express. Vamos a crear una API REST completa para gestionar un recurso de su elección.

Paso 2.1 — Configuración del proyecto Express

Inicializamos un nuevo proyecto (o reutiliza el anterior) e instalamos Express:

```
npm init -y

npm install express
```

Paso 2.2 — Escoger un dominio dominio

Eligan uno de los siguientes dominios para el API (o propongan uno de su interes):

Dominio	Recurso principal	Propiedades sugeridas
Biblioteca	`libros`	id, titulo, autor, genero, año, disponible
Recetas	`recetas`	id, nombre, ingredientes, tiempo, dificultad
Playlist Musical	`canciones`	id, titulo, artista, genero, duracion, favorita
Pokédex	`pokemon`	id, nombre, tipo, nivel, hp, ataques
Tienda	`productos`	id, nombre, precio, categoria, stock

Su recurso debe tener al menos 5 propiedades (sin contar el id).

Paso 2.3 — Endpoints obligatorios

- El API debe implementar todos los siguintes endpoints:

Método	Ruta	Respuesta
`GET`	`/`	HTML con la documentación de todos tus endpoints
`GET`	`/info`	JSON con: `mensaje`, `curso`, `tecnologia`, `version`
`GET`	`/saludo`	Texto plano con un saludo personalizado
`GET`	`/api/status`	JSON con: `ok` (boolean), `status` (string), `puerto` (number), `timestamp` (fecha actual)

- **Rutas CRUD:**

Método	Ruta	Descripción
`GET`	`/api/[recurso]`	Devuelve todos los elementos
`GET`	`/api/[recurso]/:id`	Devuelve un elemento por su ID
`POST`	`/api/[recurso]`	Crea un nuevo elemento
`PUT`	`/api/[recurso]/:id`	Actualiza un elemento existente (reemplazo completo)
`PATCH`	`/api/[recurso]/:id`	Actualiza parcialmente un elemento (solo los campos enviados)
`DELETE`	`/api/[recurso]/:id`	Elimina un elemento por su ID

La diferencia entre PUT y PATCH es importante. PUT reemplaza todo el objeto, mientras que PATCH solo modifica los campos que se envían en el body. Investigue cómo implementar ambos.

Ruta 404

Cualquier ruta que no exista debe responder con status 404 y un JSON que incluya la ruta que el usuario intentó visitar:

```
{
"error": "Ruta no encontrada",
"ruta": "/ruta-que-no-existe",
"metodo": "GET",
"sugerencia": "Visita / para ver los endpoints disponibles" }
```

Paso 2.4 — Requisitos técnicos obligatorios

El API debe cumplir con lo siguiente:

- **Datos iniciales**: Tu arreglo debe arrancar con al menos 4 elementos precargados.
- **IDs únicos**: Usa Date.now(), crypto.randomUUID(), o una librería como uid para generar identificadores únicos. No uses IDs secuenciales manuales (1, 2, 3...).
- **Validación básica en POST**: Si el body no contiene los campos obligatorios, responde con status 400 y un mensaje claro indicando qué falta.
- **Códigos HTTP correctos**:
  - 200 para respuestas exitosas
  - 201 para creación exitosa (POST)
  - 400 para datos inválidos
  - 404 para recurso no encontrado
  - 500 para errores del servidor
- **Respuestas consistentes**: Todas las respuestas JSON deben seguir una estructura predecible. Ejemplo:

```
// Éxito
{ "ok": true, "data": [ ... ] }

// Error
{ "ok": false, "error": "Mensaje descriptivo" }
```

Paso 2.5 —Filtrado por query params

Implementen al menos un filtro usando query parameters en la ruta GET principal. Por ejemplo:

- GET /api/libros?genero=ficcion
- GET /api/recetas?dificultad=facil
- GET /api/pokemon?tipo=fuego
- **Recuerden que los query parameters están disponibles en req.query:**

```
app.get("/api/libros", (req, res) => {
  const { genero } = req.query;

  if (genero) {
    const filtrados = libros.filter(
      (libro) => libro.genero.toLowerCase() === genero.toLowerCase()
    );
    return res.json({ ok: true, data: filtrados });
  }

  res.json({ ok: true, data: libros });
});
```

Criterios de Evaluación:

Rubro	Puntos
Parte 1: Todos los errores identificados, corregidos y documentados en `SOLUCION.md` con explicaciones técnicas claras	35
Parte 2: API funcional con todos los endpoints (informativos + CRUD + 404 + filtrado + validación + códigos HTTP correctos)	50
GitHub: Repositorio organizado, README descriptivo, commits frecuentes y descriptivos (mínimo 15 días de actividad)	15
Total	**100**